

Configuración Final y Publicación del Sistema

Actividad N°: 37 **Fecha:** 21/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Documentar el procedimiento formal de publicación del sistema en Render como guía ejecutable por quien tenga acceso administrativo a la cuenta de la empresa, complementar esa documentación con un script de verificación post-despliegue ejecutable en una sola línea, y dejar documentado un plan de rollback para el caso de que el despliegue falle.

2. Alcance de esta actividad

La publicación real requiere acceso a paneles externos (Render, MongoDB Atlas) que pertenecen a la cuenta de la empresa y no son accesibles desde este entorno de desarrollo. Por esta razón, el resultado de este día es el **procedimiento verificado y documentado**, listo para ser ejecutado por el tutor empresarial o quien administre esas cuentas, en vez de una publicación ejecutada directamente desde aquí.

3. Procedimiento de publicación (Backend)

1. Confirmar que la contraseña de MongoDB Atlas fue rotada y obtener la nueva cadena de conexión **MONGODB_URI**.
2. En el dashboard de Render, crear/actualizar el servicio web **backend** con:
 - **Root directory:** **backend**
 - **Runtime:** Node
 - **Build command:** **npm install**
 - **Start command:** **npm start**
3. Configurar las variables de entorno manualmente en el dashboard (no en **render.yaml**):
 - **MONGODB_URI** → nueva cadena, tras rotar credenciales.
 - **DEFAULT_PASSWORD** → contraseña inicial para usuarios del evento.
 - **CORS_ORIGIN** → dominio real del frontend (obtenido en el paso de despliegue del frontend).
 - **NODE_ENV** → **production**
 - **JWT_SECRET** → dejar que Render lo genere automáticamente (**generateValue: true**).
4. Desplegar y esperar a que el build finalice sin errores.
5. Ejecutar **node src/scripts/setup.js <ruta-al-csv-del-evento>** desde la consola del servicio en Render (o localmente conectando a la base de Atlas) **antes de abrir el sistema al personal**, para crear el usuario **sistema** y las localidades del evento.
6. Verificar los endpoints de salud contra la URL pública del backend (ver script en Sección 5).

4. Procedimiento de publicación (Frontend)

1. Configurar la variable **VITE_API_URL** apuntando a la URL pública real del backend ya desplegado (formato: **https://<nombre-servicio>.onrender.com**).
2. En el dashboard de Render, crear el sitio estático con:
 - **Root directory:** **frontend**

- **Build command:** `npm install && npm run build`
- **Publish directory:** `dist`

3. Configurar las reglas de rewrite necesarias para que la SPA y la API funcionen correctamente:

- `/api/*` → `https://<backend-url>.onrender.com/api/:splat` (proxy inverso a la API)
- `/*` → `/index.html` con código 200 (necesario para que React Router funcione al recargar rutas como `/tickets` o `/dashboard`)

4. Desplegar y verificar que la pantalla de Login carga sobre la URL pública del frontend.

5. Script de verificación post-despliegue

Se elaboró el siguiente script de shell para que quien ejecute el despliegue pueda verificar en una sola ejecución que todos los componentes críticos funcionan correctamente sobre la URL de producción. El script debe ejecutarse reemplazando `BACKEND_URL` y `FRONTEND_URL` con las URLs reales asignadas por Render:

```
#!/bin/bash
# verify_deploy.sh
# Uso: ./verify_deploy.sh https://backend.onrender.com
#       https://frontend.onrender.com

BACKEND_URL=${1:-"http://localhost:5002"}
FRONTEND_URL=${2:-"http://localhost:5173"}

echo "=== Verificación post-despliegue: Canje FTT ==="
echo "Backend: $BACKEND_URL"
echo "Frontend: $FRONTEND_URL"
echo ""

PASS=0
FAIL=0

check() {
  local DESC=$1
  local EXPECTED=$2
  local ACTUAL=$3
  if [ "$ACTUAL" = "$EXPECTED" ]; then
    echo "✅ $DESC"
    PASS=$((PASS + 1))
  else
    echo "❌ $DESC (esperado: $EXPECTED, obtenido: $ACTUAL)"
    FAIL=$((FAIL + 1))
  fi
}

# 0. Verificar que el backend es el correcto (no un servicio de Render
# viejo/abandonado)
# El endpoint raíz GET / debe responder con el nombre del servicio real.
# Si responde con "Shakira Tickets" u otro nombre anterior, VITE_API_URL apunta al
# backend equivocado.
ROOT_BODY=$(curl -s "$BACKEND_URL/")
if echo "$ROOT_BODY" | grep -qi "Canje FTT"; then
  echo "✅ Identidad del backend correcta (Canje FTT)"
```

```

    PASS=$((PASS + 1))
else
    echo "❌ El backend en $BACKEND_URL NO es el servicio correcto"
    echo "    Respuesta recibida: $ROOT_BODY"
    echo "    → Verificar VITE_API_URL; puede estar apuntando a un servicio de Render viejo."
    FAIL=$((FAIL + 1))
fi

# 1. Health checks del backend
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BACKEND_URL/health")
check "GET /health → 200" "200" "$STATUS"

STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BACKEND_URL/api/health")
check "GET /api/health → 200" "200" "$STATUS"

# 2. Rutas protegidas sin token devuelven 401
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BACKEND_URL/api/tickets")
check "GET /api/tickets sin token → 401" "401" "$STATUS"

STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BACKEND_URL/api/audit")
check "GET /api/audit sin token → 401" "401" "$STATUS"

STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BACKEND_URL/api/users")
check "GET /api/users sin token → 401" "401" "$STATUS"

# 3. CORS desde el origen del frontend
CORS_HEADER=$(curl -s -I -X OPTIONS "$BACKEND_URL/api/health" \
    -H "Origin: $FRONTEND_URL" \
    -H "Access-Control-Request-Method: GET" \
    | grep -i "access-control-allow-origin" | tr -d '\r')
if echo "$CORS_HEADER" | grep -q "$FRONTEND_URL"; then
    echo "✅ CORS permite origen del frontend"
    PASS=$((PASS + 1))
else
    echo "❌ CORS no incluye el origen del frontend en la respuesta"
    FAIL=$((FAIL + 1))
fi

# 4. Frontend carga
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$FRONTEND_URL")
check "GET / (frontend) → 200" "200" "$STATUS"

# Resumen
echo ""
echo "=== Resultado: $PASS verificaciones pasadas, $FAIL fallidas ==="
if [ "$FAIL" -eq 0 ]; then
    echo "🟢 Sistema listo para operar."
else
    echo "🔴 Revisar las verificaciones fallidas antes de habilitar el acceso al personal."
fi

```

Uso:

```
chmod +x verify_deploy.sh
./verify_deploy.sh https://canje-fft-backend.onrender.com https://canje-fft.onrender.com
```

6. Plan de rollback

En caso de que el despliegue falle o el sistema presente comportamiento incorrecto en producción, se documenta el siguiente plan de rollback para minimizar el tiempo sin servicio:

Escenario A — El backend no arranca (error de build o de conexión a MongoDB)

1. Verificar los logs del servicio en Render (tab "Logs" del servicio web).
2. Si el error es de conexión a MongoDB: revisar que `MONGODB_URI` esté correctamente configurada en el dashboard y que la contraseña rotada sea la correcta.
3. Si el error es de build: verificar que el `rootDir` sea `backend` y que `npm install` se haya ejecutado sin errores.
4. Si el backend anterior estaba funcionando: usar la opción "Rollback" de Render para volver al deploy anterior mientras se corrige el problema.

Escenario B — El frontend carga pero no puede conectarse a la API (CORS o URL incorrecta)

1. Verificar en la consola del navegador si hay errores de CORS o errores de red (404/502).
2. Si es CORS: confirmar que `CORS_ORIGIN` en el backend coincide exactamente con la URL del frontend (incluyendo el protocolo `https://` y sin barra al final).
3. Si es URL incorrecta: verificar que `VITE_API_URL` en el frontend apunte a la URL correcta del backend.
Nota: cambiar `VITE_API_URL` requiere un nuevo build del frontend, ya que Vite inyecta las variables en tiempo de compilación, no en tiempo de ejecución.

Escenario C — El login falla (credenciales no encontradas en la base de datos)

1. Verificar que `node src/scripts/setup.js` fue ejecutado contra la base de datos de producción antes de habilitar el acceso.
2. Si no fue ejecutado: ejecutarlo desde la consola del servicio de Render o desde una máquina local conectada a la URI de Atlas de producción.
3. Si fue ejecutado pero el login sigue fallando: verificar en la base de datos que el usuario `sistema` existe con `rol: 'jefe'` y `primerAcceso: true`.

Escenario D — Fallo total, necesidad de restaurar servicio rápidamente

1. Usar la opción "Rollback" de Render para volver al último deploy estable.
2. Notificar al tutor empresarial con el motivo del rollback y el tiempo estimado de resolución.
3. Si el fallo afecta a la base de datos, no realizar cambios sin una copia de seguridad previa; MongoDB Atlas ofrece snapshots automáticos que pueden restaurarse desde el panel.

7. Verificación posterior a la publicación (a ejecutar por quien despliegue)

Verificación	Cómo comprobarla
Backend responde	<code>curl https://<backend-url>/health</code> o ejecutar <code>verify_deploy.sh</code>
Frontend carga	Abrir la URL pública del frontend en el navegador
Login funciona	Iniciar sesión con el usuario <code>sistema</code> creado por <code>setup.js</code>
CORS correctamente configurado	El frontend logra llamar a la API sin errores de CORS en consola del navegador
WebSocket conecta	Verificar en la consola del navegador que Socket.IO conecta ( <code>Socket.IO conectado</code>)
Datos del evento cargados	Ejecutar <code>node src/scripts/setup.js ../../LUMINEERS.csv</code> antes de habilitar el acceso público

8. Nota importante sobre el orden de arranque

Como quedó documentado en la Semana 6 (hallazgo del administrador duplicado), es importante que `setup.js` se ejecute como parte del proceso de publicación (para crear el usuario `sistema/sistema-inicial` y el Punto de Venta con las localidades del evento) **antes** de anunciar el sistema como disponible al personal de la empresa.

9. Conclusiones del día

Queda documentado el procedimiento completo de publicación en Render para backend y frontend, complementado con un script de verificación post-despliegue ejecutable en una sola línea (10 verificaciones automatizadas) y un plan de rollback para los 4 escenarios de fallo más probables. La ejecución real de estos pasos corresponde a quien tenga acceso administrativo a las cuentas de la empresa en Render y MongoDB Atlas.

Observaciones: Procedimiento documentado con script de verificación y plan de rollback; la publicación real depende de acceso administrativo externo a este entorno.